

WPA Laboratorium 10 -- wxWidgets / events

Instrukcja

Zadania i przykłady proszę wykonywać równoległe z tekstem. Zadania proszę wykonywać jako skrypty, gdyż *Jupyter* i *wxWidgets* współpracują albo nie.

UWAGA: w dalszej części zakładamy, że biblioteka została zaimportowana w następujący sposób:

```
import wx
```

UWAGA: wx to nie jest standardowy moduł Pythona - trzeba go zainstalować w środowisku

Uwaga: Wybrane przykłady/zadania posiadają gotowe pliki umieszczone w katalogu `src`. W takim przypadku odpowiednia nazwa podana jest na początku kodu.

Uwaga: Poniższy tekst jest skrótem z dokumentacji, pomija wiele aspektów i dużo upraszcza. Na pierwsze spotkanie z *wxWidgets* może być, ale potem zalecam zapoznanie się podanymi udnośnikami.

Events - wprowadzenie

Dotychczas zbudowane aplikacje nie robię nic ciekawego po za tym że wyglądają. Biblioteka *wxWidgets*, podobnie jak inne biblioteki tego typu, jest systemem obiektowo-zdarzeniowym. To co widzieliśmy do teraz to część obiektowa. Część zdarzeniowa oparta jest na pojeciu event.

Eventy reprezentują interakcję użytkownika z interfejsem: kliknięcie w przycisk, wpisanie czegoś w pole tekstowe, wybranie opcji z menu. Eventy reprezentują również zdarzenia systemowe takie jak „tick” zegara. Wszystkie eventy skierowane do aplikacji są obsługiwane przez skomplikowany mechanizm współpracujący z system operacyjnym.

W *wxWidgets*, po wywołaniu funkcji `MainLoop()` tworzony jest specjalny obiekt event loop, który obsługuje przetwarzanie eventów. Współpracuje on z analogicznym mechanizmem, który jest częścią systemu operacyjnego, nazwijmy go `SysEventLoop`.

Dla ilustracji opiszemy (w bardzo dużym uproszczeniu) co się dzieje w przypadku kliknięcia myszką. W momencie gdy użytkownik kliknie myszką, komputer (hardware) generuje tzw. *przerwanie* (interrupt). OS ciągle czeka na przerwanie od hardware i interpretuje je za pomocą sterowników. Po interpretacji - w naszym przypadku jako kliknięcie myszką - OS dodaje do zdarzenia informacje o pozycji wskaźnika myszki na ekranie i oknie którego to dotyczy. Następnie event jest przekazywany do `SysEventLoop`. W tym momencie mamy współpracę aplikacji i OS. Aplikacja okresowo sprawdza czy są eventy do niej skierowane. W tym momencie jest - kliknięcie myszką w oknie aplikacji - więc obsługa eventu jest przekazana do aplikacji. Wewnątrz aplikacji event loop określa (na podstawie pozycji na ekranie) którego widgetu dotyczy kliknięcie i sprawdza czy do danego eventu jest stowarzyszona funkcja obsługi zdarzenia (*event handler*). Jeżeli tak to wywoływana jest ta funkcja. Zdarzenie zostało obsłużone.

Tak, jest to skomplikowane.

Programista - Ty - pisząc aplikację decydujesz jakie zdarzenia mają być obsłużone dla wybranego widgetu i pisze odpowiedni handler. Oraz dodaje do event loop tzw. event binder, który pozwala połączyć event z handlerem.

dygresja: przycisk power w komputerze nie odcina prądu, tylko też generuje przerwanie. Gdy komputer pracuje jest ono przekazywane do OS, który ma okazję bezpiecznie zakończyć pracę -- proszę sprawdzić.

W *wxWidgets*:

- event loop jest inicjalizowany po uruchomieniu `MainLoop()`
- każdy widget ma przypisane pewne rodzaje eventów (patrz dokumentacja), reprezentowane przez klasy potomne `wx.Event`
- każdy event ma określony event binder
- handler jest dowolną funkcją, typowo metoda obiektu, który odpowiada za widget
- łączenie eventu z handlerem wykonuje się za pomocą funkcji `Bind`

Aby to dobrze zrozumieć należy przeczytać kilkakrotnie dokumentację

- https://docs.wxpython.org/events_overview.html
- <https://docs.wxpython.org/wx.Event.html#wx.Event.GetEventObject>
- <https://docs.wxpython.org/wx.CommandEvent.html#wx-commandevent>

i napisać kilka programów.

My napisemy tylko kilka programów. Zaczniemy od prostej aplikacji, która:

- składa się z jednego przycisku i pola tekstowego
- każde kliknięcie przycisku powoduje zwiększenie o jeden liczby w polu tekstowym

Szkielet rozwiązania znajduje się w pliku `E11_on_button_click.py`. Zapoznaj się z kodem i uruchom go, aby sprawdzić czy wszystko działa.

Zaczniemy od handlera, jest to metoda klasy `MainFrame`. Nazwa jest dowolna, ale konwencja jest taka, aby opisywać zdarzenie które ma obsługiwać. W naszym wypadku `OnButtonClick`. Dodatkowo handler musi przyjmować dodatkowy argument - obiekt klasy `wx.Event` reprezentujący zdarzenie w naszym przypadku tendencyjnie `event`.

```
# E11_on_button_click.py

class MainFrame(wx.Frame):

    def OnButtonClick(self, event):

        self.n_clicks += 1
        self.st_counter.SetLabel( str(self.n_clicks) )
```

Zadanie handlera to zwiększyć licznik kliknięć `n_clicks` i zaktualizować wartość wyświetlaną w polu tekstowym `st_counter`.

Drugim krokiem jest zidentyfikowanie bindera do zdarzenia „kliknięcie przycisku”. Tutaj trzeba pogrzebać w [dokumentacji widgetu](#). Na szczęście `wx.Button` ma przypisany tylko jedno zdarzenie:

Events Emitted by this Class

Handlers bound for the following event types will receive a `wx.CommandEvent` parameter.

EVT_BUTTON: Process a `wxEVT_BUTTON` event, when the button is clicked.

Samo bindowanie jest proste:

```
# E11_on_button_click.py

class MainFrame(wx.Frame):

    def __init__(self)

        self.Bind(wx.EVT_BUTTON, self.OnButtonClick)
```

Teraz wszystko powinno działać, odkomentuj to co jest oznaczone (STEP1) i klikaj do woli.

Zadanie

Zauważyc, że jeżeli na przycisku jest focus, tzn jest podświetlony, to naciśnięcie Enter również generuje zdarzenie `EVT_BUTTON`

Rozbudujemy teraz naszą aplikację. Dodamy kolejny przycisk, którego zadaniem będzie zresetowanie licznika `n_clicks`.

Samo dodanie przycisku jest proste. Jednak powstaje problem: w jaki sposób rozpoznać który przycisk został kliknięty?

Potrzebujemy sposobu zidentyfikowania poszczególnych przycisków. W tym celu wykorzystamy, pmijany dotychczas parametr `id`. Jet to poprostu liczba przypisana do obiektu, jej konkretna wartość nie jest istotna. Aby ułatwić życie, w bibliotece są [predefiniowane stałe](#), które mogą być użyte do identyfikacji obiektów. My użyjemy `wx.ID_UP` oraz `wx.ID_CLEAR`.

Szkielet rozwiązania znajduje się w pliku `E12_on_button_click_2.py`. Zapoznaj się z kodem i uruchom go, aby sprawdzić czy wszystko działa. Klikając w oba przyciski widzimy, że kliknięcie obu przycisków - ku zaskoczeniu nikogo - ma ten sam efekt.

Trochę teorii: Po zbindowaniu eventu z handlerem, event loop przed wywołaniem handlera tworzy obiekt `wx.Event` dodając niego informacje o obiekcie którego zdarzenie dotyczy - w tym celu używany jest parametr `id`. Do odczytania tej informacji używamy metody `GetId`.

Zmodyfikujmy nasz handler w następujący sposób:

- wypisze on na konsoli wartości stałych `wx.ID_UP` oraz `wx.ID_CLEAR`, a także wartość zwróconą przez `GetId`

```
print( 'wx.ID_UP = {}\nwx.ID_CLEAR = {}\nGetId() ={}'.format(wx.ID_UP,wx.ID_CLEAR,event.GetId()))
```

Odkomentuj (STEP1), uruchom i obserwuj wyjście na konsoli.

Zadanie

Zmodyfikuj handler tak, aby przyciski działały zgodnie z planem.

HINT: można użyć konstrukcji if-elif-else

HINT: to dobry moment aby użyć konstrukcji match-case (patrz (STEP2))

HINT: dobrą praktyką jest obsłużenie również sytuacji w której GetId() zwraca wartość której nie chcemy, taka rezerwa.

Uwaga Z tym samym zdarzeniem może być skojarzone więcej handlerów, jeżeli chcemy umożliwić im obsłużenie zdarzenie na zakończenie można użyć funkcji Skip

Skip - This method can be used inside an event handler to control whether further event handlers bound to this event will be called after the current one returns.

Szczegóły są mocno techniczne [How Events are Processed](#)

Zadanie

Zmodyfikuj program dodając przycisk który po kliknięciu zmniejszy wartość licznika o jeden i zaktualizuje pole tekstowe.

Zadanie

Napisz program, który symuluje klawiaturę numeryczną połączoną z wyświetlaczem. Kliknięcie odpowiedniego przycisku powoduje wyświetlenie się odpowiedniego symbolu w polu tekstowym.

HINT: Patrz plik U11_lookup_table_ids.py

KOMENTARZ: parametr id nie powinien być ustalany dowolnie, aby nie wprowadzać kolizji z predefiniowanymi stałymi, tym bardziej że nie wiemy jakie są rzeczywiste wartości tych stałych w naszym systemie (moga się różnić nawet pomiędzy instalacjami).

Window identifiers are integers, and are used to uniquely determine window identity in the event system (though you can use it for other purposes). In fact, identifiers do not need to be unique across your entire application as long they are unique within the particular context you're interested in, such as a frame and its children. You may use the wx.ID_OK identifier, for example, on any number of dialogs as long as you don't have several within the same dialog.

If you pass wx.ID_ANY or -1 to a window constructor, an identifier will be generated for you automatically by wxPython. This is useful when you don't care about the exact identifier either because you're not going to process the events from the control being created or because you process the events from all controls in one place (in which case you should specify wx.ID_ANY in the wx.EvtHandler.Bind call as well). The automatically generated identifiers are always negative and so will never conflict with the user-specified identifiers which must be always positive.

See Standard event identifiers for the list of standard identifiers available.

You can use wx.ID_HIGHEST to determine the number above which it is safe to define your own identifiers. Or, you can use identifiers below wx.ID_LOWEST. Finally, you can allocate identifiers dynamically using wx.NewIdRef function too. If you use wx.NewIdRef consistently in your application, you can be sure that your identifiers don't conflict accidentally.

Z powyższego wynika, że dobrym sposobem uzyskania naszych własnych identyfikatorów jest utworzenie tzw. *lookup table*, który przechowuje jako klucze etykiety zaś jako wartości id wygenerowane przez wx.NewIdRef. Funkcja wx.NewIdRef zwraca obiekt który nie jest liczbą, ale dzięki magii programowania obiektowego można go używać jako id.

rule of thumb: Jeżeli predefiniowane id wystarczają użyj ich. Jeżeli nie, to wygeneruj za pomocą NewIdRef.

Wybrane inne kontrolki i zdarzenia

Omówimy sobie teraz wybrane kontrolki oraz zdarzenia związane z myszą i klawiaturą.

wx.TextCtrl

Dokumentacja: <https://docs.wxpython.org/wx.TextCtrl.html>

A text control allows text to be displayed and edited. It may be single line or multi-line.
Notice that a lot of methods of the text controls are found in the base wx.TextEntry class

Stworzymy małą aplikację, która pobiera wejście przez *pole tekstowe* reprezentowane przez widget wx.TextCtrl. Tekst będzie przetwarzany i wynik pojawia się w *etykiecie* reprezentowanej przez wx.StaticText. Do wywołania akcji wykorzystamy przycisk oraz zdarzenia.

Szkielet rozwiązania znajduje się w pliku E13_textctrl.py. Zapoznaj się z kodem i uruchom go, aby sprawdzić czy wszystko działa. W szczególności wpisz coś w pole tekstowe, zaznacz to, wytnij, skopiuj, wklej.

Na początek dodamy funkcjonalność polegającą na przekopiowaniu tekstu z pola tekstowego do etykiety po naciśnięciu przycisku.

Test który wpisujemy w polu tekstowym wx.TextCtrl jest przechowywany w zmiennej Value. Podstawowe funkcje, to

- GetValue Gets the contents of the control.
- SetValue Sets the new text control value. This function generates a wxEVT_TEXT event.
- ChangeValue This functions does not generate the wxEVT_TEXT event but otherwise is identical to SetValue .

Uwaga: Dostęp jest możliwy również bezpośrednio przez odwołanie do zmiennej Value - nie należy tego robić. Funkcje wymienione powyżej nazywają się *getter* oraz *setter* i poza zwykłą zmianą wartość wykonują dodatkowe czynności. Jest to ogólna zasada: jeżeli dostępne są funkcje Get i Set - korzystaj z nich; jeżeli piszesz własną klasę - napisz getter i setter.

Realizacja funkcjonalności jest bardzo prosta. Bindujemy wx.EVT_BUTTON z takim prostym handlerem:

```
def OnButtonClick(self, event):  
    self.text_output.SetLabel( self.text_input.GetValue() )
```

Zwróć uwagę na użycie settera do Label w klasie wx.StaticText i wcześniejszym kodzie. Odkomentuj (STEP1) i przetestuj.

Zadanie

Zmodyfikuj handler OnButtonClick, aby dodatkowo ustawiał wartość Wpisz jakiś tekst... w polu tekstowym.

Zmienimy teraz działanie naszego programu tak, aby tekst wpisany do pola tekstowego był natychmiast kopiowany do etykiety. W tym celu wykorzystamy event wx.EVT_TEXT, który jest generowany w chwili, gdy tekst w polu testowym jest zmieniany.

Zadanie

Spróbuj zrobić to samodzielnie. Jeżeli masz trudności odkomentuj (STEP2)

Zmodyfikujemy działanie naszego programu w taki sposób, że naciśnięcie klawisza Enter w polu tekstowym kopiowało tekst z pola do etykiety *ale wyłącznie jako duże litery*.

Najpierw uruchom program i sprawdź co się dzieje, gdy kursor znajduje się w polu tekstowym i naciśniesz Enter. (nic?)

Gdy kursor znajduje się w polu tekstowym i zostanie naciśnięty klawisz Enter generowane jest zdarzenie EVT_TEXT_ENTER

```
EVT_TEXT_ENTER:  
    Respond to a wxEVT_TEXT_ENTER event, generated when enter is pressed in a text control  
    which must have wx.TE_PROCESS_ENTER style for this event to be generated.
```

Zadanie

Spróbuj zrobić to samodzielnie. Jeżeli masz trudności odkomentuj (STEP3)

wx.MouseEvent and wx.KeyEvent

Kliknięcie myszą oraz naciśnięcie klawisza na klawiaturze powoduje wygenerowanie eventów:

wx.MouseEvent

This event class contains information about the events generated by the mouse: they include mouse buttons press and release events and mouse move events.

wx.KeyEvent

This event class contains information about key press and release events.

Na początek zajmiemy się zdarzeniami związanymi z myszą. Są one przekazywane przez klasę wx.MouseEvent

Dokumentacja:

- [MouseEvent](#)
- [MouseState](#)

Będą nas interesować następujące zdarzenia:

- EVT_LEFT_DOWN Process a wxEVT_LEFT_DOWN event. The handler of this event should normally call event.Skip() to allow the default processing to take place as otherwise the window under mouse wouldn't get the focus.
- EVT_LEFT_UP Process a wxEVT_LEFT_UP event.
- EVT_LEFT_DCLICK Process a wxEVT_LEFT_DCLICK event.
- EVT_RIGHT_DOWN Process a wxEVT_RIGHT_DOWN event.
- EVT_RIGHT_UP Process a wxEVT_RIGHT_UP event.
- EVT_MOTION Process a wxEVT_MOTION event.

Będziemy pracować z plikiem E14_mouse_events.py.

Zadanie

- Zapoznaj się z kodem
- Uruchom go.
- Klikaj w różnych miejscach okna aplikacji i obserwuj wyjście na konsoli. Co się dzieje, gdy klikasz w kolorowe kwadraciki?
- Klikaj w różnych miejscach ekranu i obserwuj wyjście na konsoli.
- Spróbuj uzyskać wszystkie 5 zdarzeń

Uwaga: Zdarzenia są przechwytywane przez widget w którym zostało zbindowane. W naszym przypadku przez wx.Frame, kliknięcie na wx.StaticText generuje zdarzenie ale nie jest ono obsługiwane przez wx.Frame. Ma to wady i zalety. Wrócimy do tego później

Zadanie

Zmodyfikuj program w następujący sposób:

- wciśnięcie lewego przycisku myszy powoduje zmianę koloru label_A na zielony
- podniesienie lewego przycisku myszy powoduje zmianę koloru label_A na pierwotny kolor

Obiekt klasy wx.MouseEvent przechowuje różne dodatkowe informacje które można uzyskać wywołując odpowiednie funkcje. Nas będzie interesować

- GetPosition() Returns the physical mouse position.
- Dragging() Returns True if this was a dragging event (motion while a button is depressed).

Zmodyfikujemy handler OnRightDown tak aby

- na konsoli pojawiała się pozycja w której kliknięto prawym klawiszem myszy
- etykieta label_B została przeniesiona do tej pozycji

w następujący sposób:

```
def OnMouseRightDown(self, event):  
  
    position = event.GetPosition()  
    print('OnMouseRightDown at {}'.format(position))  
    self.label_B.SetPosition(position)
```

Zadanie

- Odkomentuj (STEP1) i przetestuj działanie
- Co się stanie jeżeli klikniesz prawym klawiszem na kolorowym kwadracie? Dlaczego?

Zadanie

Zmodyfikuj handler OnRightUp tak aby etykieta label_A została przeniesiona do pozycji o współrzędnych x=1 y= wsp. y miejsca kliknięcia

Zadanie

Odkomentuj (STEP2). Uruchom. Wyjaśnij.

Wróćmy teraz do problemu z obiektem który jest odpowiedzialny za przetworzenie eventu. Przypomnijmy, że:

- widget do którego zostanie przypisane zdarzenie *kliknięcie myszą* jest określany na podstawie pozycji kursora na ekranie
- w naszym programie kliknięcie w kolorowy kwadrat nie wywołuje akcji (ale generuje zdarzenie)
- w naszym programie kliknięcie poza obszarem w kolorowego kwadratu wywołuje odpowiednia akcję

Dzieje się tak dlatego, że wybierany jest widget który jest „na wierzchu”, następnei wywoływana jest fucnkja zbindowana ze zdarzeniem (jeżeli taka jest)

Jest absolutnie pożądane, gdyż zapewne chcemy wejść w interakcje z tym w co klikamy. W naszym programie jednak chcemy zmieniać kolorowy kwadracik:

- zmieniać jego kolor
- przesuwać go używając czegoś podobnego do *drag-and-drop*, czyli chcemy kliknąć na obiekt, przytrzymać, przesunąć mysz i puścić. Efektem ma być przesunięcie kwadracika na nową pozycję.

Proszę sprawdzić ze kliknięcie na etykiety nie wywołuje tych akcji, ale kliknięcie poza kwadracikami już tak.

Rozwiązanie są dwa:

- zbindować event za pomocą funkcji Bind widgetu z handlerem Frame
- stworzyć klasę potomną, dodać do niej handler i tam zbindować event
- przekazać obsługę eventu z klasy Frame do App

Teraz jednak zajmiemy się ostatnim punktem.

Event może być zbindowany z wieloma funkcjami, decyzja o wywołaniu konkretnej jest podejmowana przez mechanizm opierający się na zasadach opisanych wyżej ORAZ hierarchi widgetów -- określanej przez parametr parent

Dodajmy obsługę eventu EVT_LEFT_DOWN do klasy MyApp

```
class MyApp(wx.App):  
  
    def OnInit(self):  
        """ Initialise the main GUI Application """  
  
        frame = MainFrame()  
        frame.Show()  
  
        self.Bind(wx.EVT_LEFT_DOWN, self.OnMouseLeftDown)  
  
        return True  
  
    def OnMouseLeftDown(self, event):  
        print('App catch EVT_LEFT_DOWN')
```

Zadanie

Odkomentuj (STEP3) , uruchom, klikaj obserwuj konsole. Co sie zmieniło?

Dzięki dodaniu obsługi eventu do głównego okna aplikacji kliknięcie w kolorowy kwadracik powoduje wyświetlenie odpowiedniego komunikatu. mechanizm za tym stojący jest mniej więcej taki:

- do event loop trafia zdarzenie EVT_LEFT_DOWN
- zaczyna się poszukiwanie widgetu który ma to obsłużyć - kandydatem jest ramka MainFrame a w niej label_A
- jednak label_A nie ma on zbindowanego handlera - zatem event nie jest obsłużony w RAMCE
- następuje sprawdzenie czy parent=MyApp (!) ramki może obsłużyć event okazuje się, że tak
- event jest obsługiwany w MyApp

Z drugiej strony kliknięcie w ramkę nie powoduje wyświetlenia komunikatu, gdyż event jest obsługiwany przez wytypowany obiekt.

Zadanie

Odkomentuj (STEP4) , uruchom, klikaj obserwuj konsole. Co sie zmieniło?

Użyta w tej modyfikacji funkcja Step() powoduje, że event jest przekazywany ponownie do event loop, z informacją, że programista ma intencję aby obiekt stojący wyżej w hierarchi - MyApp - również był poinformowany o zdarzeniu. To czy event zostanie obsłużony zależy od tego czy ma zbindowany handler - u nas ma.

Zadanie

Zmodyfikuj program w taki sposób, aby kliknięcie gdziekolwiek w aplikacji powodowało zmianę koloru label_B

Przejdziemy teraz do zdarzeń generowanych przez klawiaturę. Są one przekazywane przez klasę wx.KeyEvent.

Dokumentacja:

- [MouseEvent](#)
- [MouseState](#)
- [KeyEvent](#)
- [KeyCode](#)
- [KeyCategoryFlags](#)
- [KeyboardState](#)

Będą nas interesować zdarzenie:

- EVT_KEY_DOWN Process a wxEVT_KEY_DOWN event (any key has been pressed). If this event is handled and not skipped, wxEVT_CHAR will not be generated at all for this key press (but wxEVT_KEY_UP will be).

Będziemy pracować z plikiem E15_kbd_events.py.

Zadanie

- Zapoznaj się z kodem
- Uruchom go.
- Po naciskaj różne klawisze - zarówno litery, strzałki, Del etc. a na koniec Alt+F4

Zadanie

W naszym programie jest zdefiniowany handler dla zdarzenia EVT_KEY_DOWN jednak wciskanie klawiszy nie żadnego efektu. Z wyjątkiem naciśnięcia Alt+F4 które wywołało oczekiwany efekt związany z OS. Zaproponuj wyjaśnienie.

Do odczytania jaki klawisz został wcisnięty złącz dwie funkcje:

- GetKeyCode Returns the key code of the key that generated this event.
- GetUnicodeKey Returns the Unicode character corresponding to this key event.

Różnica między nimi jest następująca:

ASCII symbols return normal ASCII values, while events from special keys such as “left cursor arrow” (WXK_LEFT) return values outside of the ASCII range. See wx.KeyCode for a full list of the virtual key codes.

Note that this method returns a meaningful value only for special non-alphanumeric keys or if the user entered a Latin-1 character (this includes ASCII and the accented letters found in Western European languages but not letters of other alphabets such as e.g. Cyrillic). Otherwise it simply method returns `WXK_NONE` and `GetUnicodeKey` should be used to obtain the corresponding Unicode character.

Using `GetUnicodeKey` is in general the right thing to do if you are interested in the characters typed by the user, `GetKeyCode` should be only used for special keys (for which `GetUnicodeKey` returns `WXK_NONE`).

To handle both kinds of keys you might write:

```
def OnChar(self, event):

    keycode = event.GetUnicodeKey()

    if keycode != wx.WXK_NONE:

        # It's a printable character

    else:

        # It's a special key, deal with all the known ones:
        keycode = event.GetKeyCode()
        if keycode in [wx.WXK_LEFT, wx.WXK_RIGHT]:
            # move cursor ...
            MoveCursor()

        elif keycode == wx.WXK_F1:
            # give help ...
            GiveHelp()
```

Zadanie

- Odkomentuj (STEP1), zapoznaj się z kodem i przetestuj.
- Zauważ, że naciśnięcie klawiszy Alt, Shift oraz Ctrl także ma przypisaną wartość
- Zauważ, że naciśnięcie kombinacji Ctrl+A zwraca kod odpowiadający A

Jednym ze sposobów odczytania kombinacji typu Ctrl+A jest użycie pary funkcji

- `HasAnyModifiers()` zwraca True jeżeli użyto Alt, Shift oraz Ctrl oaz innego znaku
- `GetModifiers()` zwraca informację (maskę bitową) odpowiadającą danemu klawiszowi: `wx.MOD_ALT`, `wx.MOD_CONTROL`, `wx.MOD_SHIFT`

Zadanie

- Odkomentuj (STEP2), zapoznaj się z kodem i przetestuj.
- Zauważ, że naciśnięcie kombinacji Ctrl/Alt/Shift+A oraz Ctrl+Shift+A wywołyje efekt

Zadanie

Zmodyfikuj program tak aby

- kombinacji klawiszy Alt+1,...,Alt+0 zmieniał kolor `label_A` na inny
- użycie strzałek przemieszczało `label_B`
- naciśnięcie Esc przywracało stan początkowy

TextCtrl and MouseEvent

Wróćmy do przechwytywania eventów przez widgety.

Będziemy pracować z plikiem `E16_custom_textctrl.py`.

Zadanie

- Zapoznaj się z kodem
- Uruchom go.
- Klikaj w różnych miejscach okna aplikacji aż do sprawdzenia wszystkich zaimplementowanych funkcjonalności.

Nasz program zachowuje się w ten sposób że wstawiając kursor w polu tekstowym, musimy skasować poprzedni tekst aby wpisać nowy. Zmienimy jego zachowanie aby wstawienie kursora do pola tekstowego wyczyściło zawartość.

Można to uzyskać na dwa sposoby:

- Używając funkcji `Bind` **widgetu** do połączenia eventu z handlerem ramki
- Stworzyć klasę potomną od `wx.TextCtrl` i tam obsłużyć event

W pierwszym przypadku robi się to tak (odkomentuj (STEP1))

```
self.text_input.Bind(wx.EVT_LEFT_DOWN, self.ClearText)

def ClearText(self, event):
    self.text_input.SetValue( '' )
```

Drugi sposób jesteś w stanie zaimplementować wykonać samodzielnie

Zadanie

Stwórz klasę potomną od `wx.TextCtrl`, która w przypadku kliknięcia

- lewym klawiszem myszy wyczyści zawartość pola tekstowego.
- lewym klawiszem myszy umieszcza tam napis `Wpisz jakiś tekst...`

HINT patrz (STEP2)

Zadanie

Jeżeli masz ochotę, to opanuj obsługę widgetów:

- `wx.StaticLine`
- `wx.StaticBox`
- `wx.ToggleButton/wx.RadioButton/wx.BitmapButton`
- `wx.ListBox`
- `wx.CheckBox/wx.RadioButton`
- `wx.Choice`
- `wx.Combobox`
- `wx.Slider`
- `wx.SpinButton/wx.SpincCtrl`
- `wx.StaticBitmap`

Dialogs

The generic `wx.Dialog` class can be used to build any custom dialog you require. It can be used to create modal and modeless dialogs:

- A modal dialog blocks program flow and user input on other windows until it is dismissed.
- A modeless dialog behaves more like a frame in that program flow continues, and input in other windows is still possible.

The `wx.Dialog` class provides two versions of the `show` method to support modal and modeless dialogs. The `ShowModal()` method is used to display a modal dialog, while the `Show()` is used to show a modeless dialog. As well as the generic `wx.Dialog` class, the `wxPython` library provides numerous prebuilt dialogs for common situations. These pre built dialogs include:

- `wx.ColourDialog` This class is used to generate a colour chooser dialog.
- `wx.DirDialog` This class provides a directory chooser dialog.
- `wx.FileDialog` This class provides a file chooser dialog.
- `wx.FontDialog` This class provides a font chooser dialog.
- `wx.MessageDialog` This class can be used to generate a single or multi-line message or information dialog. It can support Yes, No and Cancel options. It can be used for generic messages or for error messages.
- `wx.MultiChoiceDialog` This dialog can be used to display a list of strings and allows the user to select one or more values for the list.
- `wx.PasswordEntryDialog` This class represents a dialog that allows a user to enter a one-line password string from the user.
- `wx.ProgressDialog` If supported by the GUI platform, then this class will provide the platform's native progress dialog, otherwise it will use the pure Python `wx.GenericProgressDialog`. The `wx.GenericProgressDialog` shows a short message and a progress bar.
- `wx.TextEntryDialog` This class provides a dialog that requests a one-line text string from the user.

Most of the dialogs that return a value follow the same pattern. This pattern returns a value from the `ShowModal()` method that indicates if the user selected OK or CANCEL (using the return value `wx.ID_OK` or `wx.ID_CANCEL`). The selected/entered value can then be obtained from a suitable `get` method such as `GetColourData()` for the `ColourDialog` or `GetPath()` for the `DirDialog`.